AMRITA VISHWA VIDYAPEETHAM CHENNAI CAMPUS

Amrita School of Engineering

Department of ECE

Coding Practice

Activity Sheets - 03

(For the Batch 2023-2027)

Name: N.Lokeshwar Varma
Roll No.: CH.EN.U4CCE24020
Department: CCE

Activity Sheet 03 **Start Date12/05/2026 End Date: 15/05/2026**

Topic: Strings (Easy + Medium + Hard Coding Questions]

Instructions

The Mock Test syllabus for CSE CT | CYS | AIE | RAI program is enclosed in the attached document for your reference.

Materials for the Mock Test[Must-DO Coding Questions] is provided in a link.

It is mandatory to upload the completed Activity sheets before the deadline in the MS-Form link shared by your class Advisor.

Activity Sheet: 03 - Coding Questions from Strings – Part 1 [Easy + Medium +Hard Questions]

Each Activity Sheets contains total of 20 Questions.

Platform: Any Online Compiler of your choice – GDB Compiler

Language: C | C++ | JAVA | PYTHON

The Activity Sheet will have the coding Questions and the Test Case. Once executed in Online Compiler, take screen shot of the code and the output and paste it in the space provided in the activity sheet.

The Final Activity Sheet should be in PDF Format and the file name should be your roll number.

Course Instructors: CARE TEAM
Dr.R.Annamalai Assistant Professor (Sl.Grade) |CSE-AIE
Dr.J.Umamageswaran Assistant Professor (Sr.Grade) | CSE-CT
Ms.D. Sasikala Assistant Professor | CSE-AIE
Mr.NirmalRaj Assistant Professor | CSE-CYS

1. Given two strings A and B. Find the minimum number of steps required to transform string A into string B. The only allowed operation for the transformation is selecting a character from string A and inserting it in the beginning of string A.

Example 1:

Input:

A = "abd"

B = "bad"

Output: 1

Explanation: The conversion can take place in1 operation: Pick 'b' and place it at the front.

Example 2:

Input:

A = "GeeksForGeeks"

B = "ForGeeksGeeks"

Output: 3

Explanation: The conversion can take place in 3 operations: Pick 'r' and place it at the front.

A = "rGeeksFoGeeks"

Pick 'o' and place it at the front.

A = "orGeeksFGeeks"

Pick 'F' and place it at the front.

A = "ForGeeksGeeks"

Your Task:

You dont need to read input or print anything. Complete the function transform() which takes two strings A and B as input parameters and returns the minimum number of steps required to transform A into B. If transformation is not possible return -1.

Expected Time Complexity: O(N) where N is max(length of A, length of B)

Expected Auxiliary Space: O(1)

Constraints:

1<= A.length(), B.length() <= 104

Code:

```c
#include <stdio.h>
#include <string.h>

int transform(char A[], char B[])
{
    int lenA = strlen(A);
    int lenB = strlen(B);

    if(lenA != lenB)
    {
        return -1;
    }

    int countA[256] = {0};
    int countB[256] = {0};

    // Check if both strings contain same characters
    for(int i = 0; i < lenA; i++)
    {
        countA[A[i]]++;
        countB[B[i]]++;
    }

    for(int i = 0; i < 256; i++)
    {
        if(countA[i] != countB[i])
        {
            return -1;
        }
    }

    int i = lenA - 1;
    int j = lenB - 1;
    int steps = 0;

    while(i >= 0)
    {
        if(A[i] == B[j])
        {
            i--;
            j--;
        }
        else
        {
            steps++;
            i--;
        }
    }

    return steps;
}

int main()
{
    char A[10000], B[10000];

    printf("Roll Number : CH.EN.U4CCE24020\n\n");

    printf("Enter string A: ");
    scanf("%s", A);

    printf("Enter string B: ");
    scanf("%s", B);
```

Output:

```
Roll Number : CH.EN.U4CCE24020

Enter string A: "abd"
Enter string B: "bad"

Minimum number of steps: 2

--------------------------------
Process exited after 30.73 seconds with return value 0
Press any key to continue . . .
```

```
Roll Number : CH.EN.U4CCE24020

Enter string A: "GeeksForGeeks"
Enter string B: "ForGeeksGeeks"

Minimum number of steps: 4

--------------------------------
Process exited after 25.47 seconds with return value 0
Press any key to continue . . .
```

2. Given two non-empty strings s1 and s2, consisting only of lowercase English letters, determine whether they are anagrams of each other or not.

Two strings are considered anagrams if they contain the same characters with exactly the same frequencies, regardless of their order.

Examples:

Input: s1 = "geeks" s2 = "kseeg"

Output: true

Explanation: Both the string have same characters with same frequency. So, they are anagrams.

Input: s1 = "allergy", s2 = "allergyy"

Output: false

Explanation: Although the characters are mostly the same, s2 contains an extra 'y' character. Since the frequency of characters differs, the strings are not anagrams.

Constraints:

$1 \leq$ s1.size(), s2.size() ≤ 105

s1, s2 consists of lowercase English letters.

Code:

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char s1[100000], s2[100000];

    printf("Roll Number : CH.EN.U4CCE24020\n\n");

    printf("Enter first string: ");
    scanf("%s", s1);

    printf("Enter second string: ");
    scanf("%s", s2);

    int len1 = strlen(s1);
    int len2 = strlen(s2);

    if(len1 != len2)
    {
        printf("\nOutput: false\n");
        return 0;
    }

    int freq1[26] = {0};
    int freq2[26] = {0};

    for(int i = 0; i < len1; i++)
    {
        freq1[s1[i] - 'a']++;
        freq2[s2[i] - 'a']++;
    }

    int isAnagram = 1;

    for(int i = 0; i < 26; i++)
    {
        if(freq1[i] != freq2[i])
        {
            isAnagram = 0;
            break;
        }
    }

    if(isAnagram)
    {
        printf("\nOutput: true\n");
    }
    else
    {
        printf("\nOutput: false\n");
    }

    return 0;
}
```

Output

```
Roll Number : CH.EN.U4CCE24020

Enter first string: "geeks"
Enter second string: "kseeg"

Output: true

--------------------------------
Process exited after 23.29 seconds with return value 0
Press any key to continue . . .
```

```
Roll Number : CH.EN.U4CCE24020

Enter first string: "allergy
Enter second string:  "allergyy"

Output: false

--------------------------------
Process exited after 19.65 seconds with return value 0
Press any key to continue . . .
```

3. Find the count of all possible strings of size n.Each character of the string is either 'R', 'B' or 'G'. In the final string there needs to be at least r number of 'R', at least b number of 'B' and at least g number of 'G' (such that r + g + b <= n).

Example 1:

Input: n = 4, r = 1, g = 1, b = 1

Output: 36

Explanation: No. of 'R' >= 1, No. of 'G' >= 1, No. of 'B' >= 1 and (No. of 'R') + (No. of 'B') + (No. of 'G') = n then following cases are possible:

1. RBGR and its 12 permutation
2. RBGB and its 12 permutation
3. RBGG and its 12 permutation

Hence answer is 36.

Example 2:

Input: n = 4, r = 2, g = 0, b = 1

Output: 22

Explanation: No. of 'R' >= 2, No. of 'G' >= 0, No. of 'B' >= 1 and (No. of 'R') + (No. of 'B')+ (No. of 'G') <= n then following cases are possible:

1. RRBR and its 4 permutation
2. RRBG and its 12 permutation
3. RRBB and its 6 permutation

Hence answer is 22.

Your Task:

You dont need to read input or print anything. Complete the function possibleStrings() which takes n, r, g, b as input parameter and returns the count of number of all possible strings..

Expected Time Complexity: O(n2)

Expected Auxiliary Space: O(n)

Constraints:

Activity Sheet 03 **Start Date12/05/2026 End Date: 15/05/2026**

Topic: Strings (Easy + Medium + Hard Coding Questions]

1<= n <=20

1<= r+b+g <=n

Code

```c
#include <stdio.h>

long long factorial(int n)
{
    long long fact = 1;

    for(int i = 1; i <= n; i++)
    {
        fact *= i;
    }

    return fact;
}

long long possibleStrings(int n, int r, int g, int b)
{
    long long count = 0;

    for(int i = r; i <= n; i++)
    {
        for(int j = g; j <= n; j++)
        {
            for(int k = b; k <= n; k++)
            {
                if(i + j + k == n)
                {
                    long long ways;

                    ways = factorial(n) /
                           (factorial(i) * factorial(j) * factorial(k));

                    count += ways;
                }
            }
        }
    }

    return count;
}

int main()
{
    int n, r, g, b;

    printf("Roll Number : CH.EN.U4CCE24020\n\n");

    printf("Enter value of n: ");
    scanf("%d", &n);

    printf("Enter minimum number of R: ");
    scanf("%d", &r);

    printf("Enter minimum number of G: ");
    scanf("%d", &g);
```

Output

```
Roll Number : CH.EN.U4CCE24020

Enter value of n: 4
Enter minimum number of R: 1
Enter minimum number of G: 1
Enter minimum number of B: 1

Total possible strings: 36

--------------------------------
Process exited after 15.26 seconds with return value 0
Press any key to continue . . .
```

```
Roll Number : CH.EN.U4CCE24020

Enter value of n: 4
Enter minimum number of R: 2
Enter minimum number of G: 0
Enter minimum number of B: 1

Total possible strings: 22

--------------------------------
Process exited after 9.913 seconds with return value 0
Press any key to continue . . .
```

4. Given a valid expression containing only binary operators '+', '-', '*', '/' and operands, remove all the redundant parenthesis.

A set of parenthesis is said to be redundant if, removing them, does not change the value of the expression.

Note: The operators '+' and '-' have the same priority. '*' and '/' also have the same priority. '*' and '/' have more priority than '+' and '-'.

Example 1:

Input:

Exp = (A*(B+C))

Output: A*(B+C)

Explanation: The outermost parenthesis

are redundant.

Example 2:

Input:

Exp = A+(B+(C))

Output: A+B+C

Explanation: All the parenthesis

are redundant.

Your Task:

You don't need to read input or print anything. Your task is to complete the function removeBrackets() which takes the string Exp as input parameters and returns the updated expression.

Expected Time Complexity: O(N)

Expected Auxiliary Space: O(N)

Constraints:

1 < Length of Exp < 105

Exp contains uppercase english letters, '(' , ')', '+', '-', '*' and '/'.

Code:

```c
#include <stdio.h>
#include <string.h>

int isRedundant(char exp[], int start, int end)
{
    int hasLowPriority = 0;

    for(int i = start + 1; i < end; i++)
    {
        if(exp[i] == '+' || exp[i] == '-')
        {
            hasLowPriority = 1;
        }
    }

    // Check previous and next operators
    char prev = (start > 0) ? exp[start - 1] : ' ';
    char next = (end < strlen(exp) - 1) ? exp[end + 1] : ' ';

    if((prev == '*' || prev == '/') && hasLowPriority)
    {
        return 0;
    }

    if((next == '*' || next == '/') && hasLowPriority)
    {
        return 0;
    }

    return 1;
}

int main()
{
    char Exp[100000];

    printf("Roll Number : CH.EN.U4CCC24020\n\n");

    printf("Enter the expression: ");
    scanf("%s", Exp);

    int n = strlen(Exp);

    int remove[100000] = {0};
    int stack[100000];
    int top = -1;

    for(int i = 0; i < n; i++)
    {
        if(Exp[i] == '(')
        {
            stack[++top] = i;
        }
        else if(Exp[i] == ')')
        {
            int start = stack[top--];
            int end = i;

            if(isRedundant(Exp, start, end))
            {
                remove[start] = 1;
                remove[end] = 1;
            }
        }
    }

    printf("\nExpression after removing redundant brackets:\n");

    for(int i = 0; i < n; i++)
    {
        if(remove[i] == 0)
        {
            printf("%c", Exp[i]);
        }
    }

    printf("\n");

    return 0;
}
```

Output

```
Roll Number : CH.EN.U4CCE24020

Enter the expression: (A*(B+C))

Expression after removing redundant brackets:
A*(B+C)

--------------------------------
Process exited after 15.46 seconds with return value 0
Press any key to continue . . .
```

```
Roll Number : CH.EN.U4CCE24020

Enter the expression: A+(B+(C))

Expression after removing redundant brackets:
A+B+C

--------------------------------
Process exited after 11.53 seconds with return value 0
Press any key to continue . . .
```

5. Find the number of unique words consisting of lowercase alphabets only of length N that can be formed with at-most K contiguous vowels.

Example 1:

Input:
N = 2
K = 0
Output:
441
Explanation:
Total of 441 unique words are possible
of length 2 that will have K(=0) vowels
together, e.g. "bc", "cd", "df", etc are
valid words while "ab" (with 1 vowel) is
not a valid word.

Example 2:

Input:
N = 1
K = 1
Output:
26
Explanation:
All the english alphabets including
vowels and consonants; as atmost K(=1)
vowel can be taken.

Your Task:
You don't need to read input or print anything. Your task is to complete the function kvowelwords() which takes an Integer N, an intege K and returns the total number of words of size N with atmost K vowels. As the answer maybe to large please return answer modulo 1000000007.

Expected Time Complexity: O(N*K)
Expected Auxiliary Space: O(N*K)

Constraints:
$1 \leq N \leq 1000$
$0 \leq K \leq N$

Code:

```c
#include <stdio.h>

#define MOD 1000000007

long long kvowelwords(int N, int K)
{
    long long dp[1001][1001] = {0};

    dp[0][0] = 1;

    for(int i = 1; i <= N; i++)
    {
        // Add consonant
        long long sum = 0;

        for(int j = 0; j <= K; j++)
        {
            sum = (sum + dp[i - 1][j]) % MOD;
        }

        dp[i][0] = (sum * 21) % MOD;

        // Add vowel
        for(int j = 1; j <= K; j++)
        {
            dp[i][j] = (dp[i - 1][j - 1] * 5) % MOD;
        }
    }

    long long answer = 0;

    for(int j = 0; j <= K; j++)
    {
        answer = (answer + dp[N][j]) % MOD;
    }

    return answer;
}

int main()
{
    int N, K;

    printf("Roll Number : CH.EN.U4CCE24020\n\n");

    printf("Enter value of N: ");
    scanf("%d", &N);

    printf("Enter value of K: ");
    scanf("%d", &K);

    long long result = kvowelwords(N, K);

    printf("\nTotal unique words: %lld\n", result);

    return 0;
}
```

Output

```
Roll Number : CH.EN.U4CCE24020

Enter value of N: 1
Enter value of K: 1

--------------------------------
Process exited after 9.01 seconds with return value 3221225725
Press any key to continue . . .
```

6. Problem Statement – Given a string S(input consisting) of '*' and '#'. The length of the string is variable. The task is to find the minimum number of '*' or '#' to make it a valid string. The string is considered valid if the number of '*' and '#' are equal. The '*' and '#' can be at any position in the string.

Note : The output will be a positive or negative integer based on number of '*' and '#' in the input string.

- (*>#): positive integer
- (#>*): negative integer
- (#=*): 0

Code:

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char S[1000];

    printf("Roll Number : CH.EN.U4CCE24020\n\n");

    printf("Enter the string containing '*' and '#': ");
    scanf("%s", S);

    int star = 0;
    int hash = 0;

    for(int i = 0; i < strlen(S); i++)
    {
        if(S[i] == '*')
        {
            star++;
        }
        else if(S[i] == '#')
        {
            hash++;
        }
    }

    int result = star - hash;

    printf("\nOutput: %d\n", result);

    return 0;
}
```

Output

```
Roll Number : CH.EN.U4CCE24020

Enter the string containing '*' and '#': sam*

Output: 1

--------------------------------
Process exited after 10.47 seconds with return value 0
Press any key to continue . . .
```

7. Given a string s, reverse the string without reversing its individual words. Words are separated by dots(.).

Note: The string may contain leading or trailing dots(.) or multiple dots(.) between two words. The returned string should only have a single dot(.) separating the words, and no extra dots should be included.

Examples :

Input: s = "i.like.this.program.very.much"
Output: "much.very.program.this.like.i"
Explanation: The words in the input string are reversed while maintaining the dots as separators, resulting in "much.very.program.this.like.i".
Input: s = "..geeks..for.geeks."
Output: "geeks.for.geeks"
Explanation: After removing extra dots and reversing the whole string, the input string becomes "geeks.for.geeks".
Explanation: The input string contains only one word with extra dots around it. After removing the extra dots, the output is "home".
Constraints:
$1 \leq s.length() \leq 106$
String s contains only lowercase English alphabets and dots.

Code:

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char s[100000];
    char words[1000][1000];

    printf("Roll Number : CH.EN.U4CCE24020\n\n");

    printf("Enter the string:\n");
    scanf("%s", s);

    int len = strlen(s);
    int count = 0;
    int k = 0;

    char temp[1000];

    // Extract words
    for(int i = 0; i <= len; i++)
    {
        if(s[i] != '.' && s[i] != '\0')
        {
            temp[k++] = s[i];
        }
        else
        {
            if(k > 0)
            {
                temp[k] = '\0';
                strcpy(words[count++], temp);
                k = 0;
            }
        }
    }

    printf("\nReversed string:\n");

    // Print words in reverse order
    for(int i = count - 1; i >= 0; i--)
    {
        printf("%s", words[i]);

        if(i != 0)
        {
            printf(".");
        }
    }

    printf("\n");

    return 0;
}
```

Output

```
Roll Number : CH.EN.U4CCE24020

Enter the string:
"i.like.this.program.very.much"

Reversed string:
much".very.program.this.like."i

--------------------------------
Process exited after 17.08 seconds with return value 0
Press any key to continue . . .
```

```
Roll Number : CH.EN.U4CCE24020

Enter the string:
"..geeks..for.geeks."

Reversed string:
".geeks.for.geeks."

--------------------------------
Process exited after 8.414 seconds with return value 0
Press any key to continue . . .
```

8. A City Bus is a Ring Route Bus which runs in circular fashion.That is, Bus once starts at the Source Bus Stop, halts at each Bus Stop in its Route and at the end it reaches the Source Bus Stop again.
If there are n number of Stops and if the bus starts at Bus Stop 1, then after nth Bus Stop, the next stop in the Route will be Bus Stop number 1 always.
If there are n stops, there will be n paths.One path connects two stops. Distances (in meters) for all paths in Ring Route is given in array Path[] as given below:
Path = [800, 600, 750, 900, 1400, 1200, 1100, 1500]
Fare is determined based on the distance covered from source to destination stop as Distance between Input Source and Destination Stops can be measured by looking at values in array Path[] and fare can be calculated as per following criteria:

- If d =1000 metres, then fare=5 INR
- (When calculating fare for others, the calculated fare containing any fraction value should be ceiled. For example, for distance 900n when fare initially calculated is 4.5 which must be ceiled to 5)

Path is circular in function. Value at each index indicates distance till current stop from the previous one. And each index position can be mapped with values at same index in BusStops [] array, which is a string array holding abbreviation of names for all stops as-
“THANERAILWAYSTN” = ”TH”, “GAONDEVI” = “GA”, “ICEFACTROY” = “IC”,
“HARINIWASCIRCLE” = “HA”, “TEENHATHNAKA” = “TE”, “LUISWADI” = “LU”,
“NITINCOMPANYJUNCTION” = “NI”, “CADBURRYJUNCTION” = “CA”
Given, n=8, where n is number of total BusStops.
BusStops = [“TH”, ”GA”, ”IC”, ”HA”, ”TE”, ”LU”, ”NI”,”CA”]
Write a code with function getFare(String Source, String Destination) which take Input as source and destination stops(in the format containing first two characters of the Name of the Bus Stop) and calculate and return travel fare.

Code:

```c
#include <stdio.h>
#include <string.h>
#include <math.h>

int getIndex(char stop[][3], int n, char name[])
{
    for(int i = 0; i < n; i++)
    {
        if(strcmp(stop[i], name) == 0)
        {
            return i;
        }
    }

    return -1;
}

int main()
{
    char BusStops[8][3] = {"TH", "GA", "IC", "HA", "TE", "LU", "NI", "CA"};

    int Path[8] = {800, 600, 750, 900, 1400, 1200, 1100, 1500};

    char Source[3], Destination[3];

    printf("Roll Number : CH.EN.U4CCE24020\n\n");

    printf("Available Stops:\n");
    printf("TH GA IC HA TE LU NI CA\n\n");

    printf("Enter Source Stop: ");
    scanf("%s", Source);

    printf("Enter Destination Stop: ");
    scanf("%s", Destination);

    int sourceIndex = getIndex(BusStops, 8, Source);
    int destIndex = getIndex(BusStops, 8, Destination);

    if(sourceIndex == -1 || destIndex == -1)
    {
        printf("\nInvalid Bus Stop\n");
        return 0;
    }

    int distance = 0;
    int i = sourceIndex;

    // Calculate circular distance
    while(i != destIndex)
    {
        distance += Path[i];
        i = (i + 1) % 8;
    }
```

```c
    {
        distance += Path[i];
        i = (i + 1) % 8;
    }

    // Fare calculation
    double fare = (double)distance * 5 / 1000;

    int finalFare = ceil(fare);

    printf("\nDistance Travelled: %d meters\n", distance);
    printf("Travel Fare: %d INR\n", finalFare);

    return 0;
}
```

Output

```
Roll Number : CH.EN.U4CCE24020

Available Stops:
TH GA IC HA TE LU NI CA

Enter Source Stop: TH
Enter Destination Stop: LU

Distance Travelled: 4450 meters
Travel Fare: 23 INR

--------------------------------
Process exited after 42.04 seconds with return value 0
Press any key to continue . . .
```

9. A new alien language uses the English alphabet, but the order of letters is unknown. You are given a list of words[] from the alien language's dictionary, where the words are claimed to be sorted lexicographically according to the language's rules.

Your task is to determine the correct order of letters in this alien language based on the given words. If the order is valid, return a string containing the unique letters in lexicographically increasing order as per the new language's rules. If there are multiple valid orders, return any one of them.

However, if the given arrangement of words is inconsistent with any possible letter ordering, return an empty string ("").

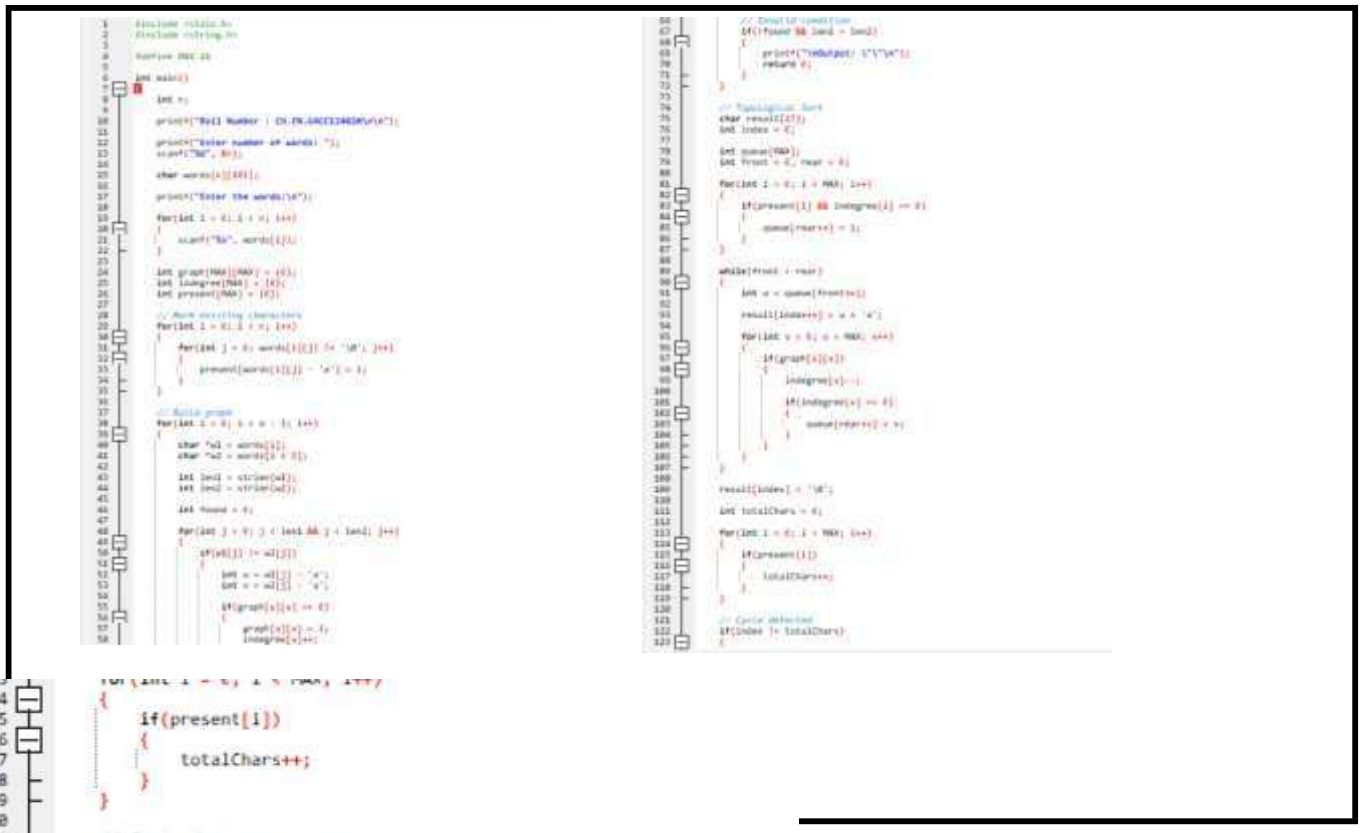
A string a is lexicographically smaller than a string b if, at the first position where they differ, the character in a appears earlier in the alien language than the corresponding character in b. If all characters in the shorter word match the beginning of the longer word, the shorter word is considered smaller.

Note: Your implementation will be tested using a driver code. It will print true if your returned order correctly follows the alien language's lexicographic rules; otherwise, it will print false.

Examples:

Input: words[] = ["baa", "abcd", "abca", "cab", "cad"]
Output: true
Explanation: A possible correct order of letters in the alien dictionary is "bdac".
The pair "baa" and "abcd" suggests 'b' appears before 'a' in the alien dictionary.
The pair "abcd" and "abca" suggests 'd' appears before 'a' in the alien dictionary.
The pair "abca" and "cab" suggests 'a' appears before 'c' in the alien dictionary.
The pair "cab" and "cad" suggests 'b' appears before 'd' in the alien dictionary.
So, 'b' → 'd' → 'a' → 'c' is a valid ordering.
Input: words[] = ["caa", "aaa", "aab"]
Output: true
Explanation: A possible correct order of letters in the alien dictionary is "cab".
The pair "caa" and "aaa" suggests 'c' appears before 'a'.
The pair "aaa" and "aab" suggests 'a' appear before 'b' in the alien dictionary.
So, 'c' → 'a' → 'b' is a valid ordering.
Input: words[] = ["ab", "cd", "ef", "ad"]
Output: ""
Explanation: No valid ordering of letters is possible.
The pair "ab" and "ef" suggests "a" appears before "e".
The pair "ef" and "ad" suggests "e" appears before "a", which contradicts the ordering rules.
Constraints:
1 ≤ words.length ≤ 500
1 ≤ words[i].length ≤ 100
words[i] consists only of lowercase English letters.

Code:



```
    {
        if(present[i])
        {
            totalChars++;
        }
    }

    // Cycle detected
    if(index != totalChars)
    {
        printf("\nOutput: \"\"\n");
    }
    else
    {
        printf("\nPossible Alien Dictionary Order: %s\n", result);
    }

    return 0;
}
```

Output

```
Roll Number : CH.EN.U4CCE24020

Enter number of words: ["baa", "abcd", "abca", "cab", "cad"]

--------------------------------
Process exited after 15.69 seconds with return value 3221225725
```

10. Jack and Jill are playing a string game. Jack has given Jill two strings A and B. Jill has to derive a string C from A, by deleting elements from string A, such that string C does not contain any element of string

B. Jill needs help to do this task. She wants a program to do this as she is lazy. Given strings A and B as

input, give string C as Output.

Code:

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char A[1000], B[1000];

    printf("Roll Number : CH.EN.U4CCE24020\n\n");

    printf("Enter string A: ");
    scanf("%s", A);

    printf("Enter string B: ");
    scanf("%s", B);

    int present[256] = {0};

    // Mark characters present in B
    for(int i = 0; i < strlen(B); i++)
    {
        present[B[i]] = 1;
    }

    char C[1000];
    int k = 0;

    // Form string C
    for(int i = 0; i < strlen(A); i++)
    {
        if(present[A[i]] == 0)
        {
            C[k++] = A[i];
        }
    }

    C[k] = '\0';

    printf("\nOutput String C: %s\n", C);

    return 0;
}
```

Output

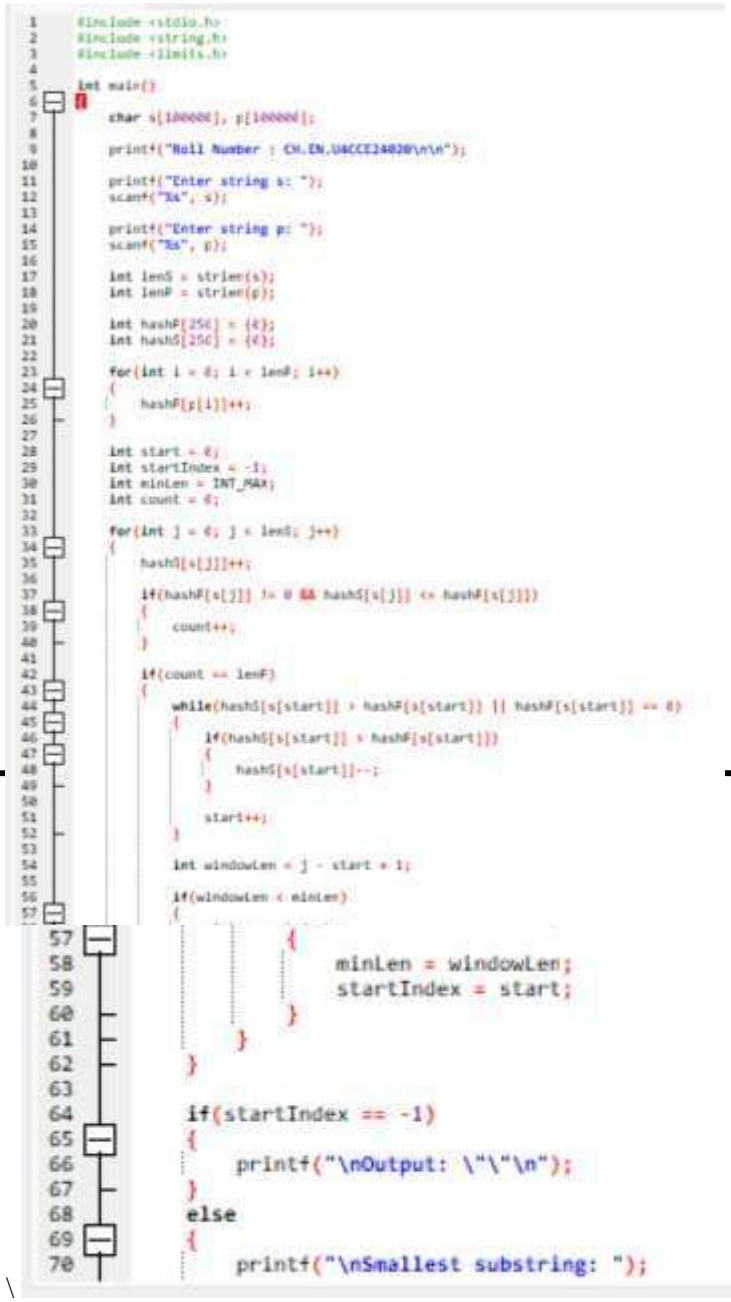
```
Roll Number : CH.EN.U4CCE24020

Enter string A: computer
Enter string B: cat

Output String C: ompuer

--------------------------------
Process exited after 38.43 seconds with return value 0
Press any key to continue . . .
```

11. Given two strings s and p. Find the smallest substring in s consisting of all the characters (including duplicates) of the string p. Return empty string in case no such substring is present.
If there are multiple such substring of the same length found, return the one with the least starting index.

Examples:

Input: s = "timetopractice", p = "toc"
Output: "toprac"
Explanation: "toprac" is the smallest substring in which "toc" can be found.
Input: s = "zoomlazapzo", p = "oza"
Output: "apzo"
Explanation: "apzo" is the smallest substring in which "oza" can be found.
Constraints:
$1 \leq$ s.length(), p.length() ≤ 106
s, p consists of lowercase english letters

Code:



```c
#include <stdio.h>
#include <string.h>
#include <limits.h>

int main()
{
    char s[100000], p[100000];

    printf("Roll Number : CH.EN.U4CCE24020\n\n");

    printf("Enter string s: ");
    scanf("%s", s);

    printf("Enter string p: ");
    scanf("%s", p);

    int lenS = strlen(s);
    int lenP = strlen(p);

    int hashP[256] = {0};
    int hashS[256] = {0};

    for(int i = 0; i < lenP; i++)
    {
        hashP[p[i]]++;
    }

    int start = 0;
    int startIndex = -1;
    int minLen = INT_MAX;
    int count = 0;

    for(int j = 0; j < lenS; j++)
    {
        hashS[s[j]]++;

        if(hashP[s[j]] != 0 && hashS[s[j]] <= hashP[s[j]])
        {
            count++;
        }

        if(count == lenP)
        {
            while(hashS[s[start]] > hashP[s[start]] || hashP[s[start]] == 0)
            {
                if(hashS[s[start]] > hashP[s[start]])
                {
                    hashS[s[start]]--;
                }

                start++;
            }

            int windowLen = j - start + 1;

            if(windowLen < minLen)
            {
                minLen = windowLen;
                startIndex = start;
            }
        }
    }

    if(startIndex == -1)
    {
        printf("\nOutput: \"\"\n");
    }
    else
    {
        printf("\nSmallest substring: ");
```

\

Output

```
Roll Number : CH.EN.U4CCE24020

Enter string s: "timetopractice"
Enter string p: "toc"

Smallest substring: "timetopractice"

--------------------------------
Process exited after 23.33 seconds with return value 0
Press any key to continue . . .
```

```
Roll Number : CH.EN.U4CCE24020

Enter string s: "zoomlazapzo"
Enter string p: "oza"

Smallest substring: "zoomlazapzo"

--------------------------------
Process exited after 24.46 seconds with return value 0
Press any key to continue . . .
```

12. Given two strings s1 and s2. Return the minimum number of operations required to convert s1 to s2.
The possible operations are permitted:

Insert a character at any position of the string.
Remove any character from the string.
Replace any character from the string with any other character.
Examples:

Input: s1 = "geek", s2 = "gesek"
Output: 1
Explanation: One operation is required, inserting 's' between two 'e' in s1.
Constraints:
$1 \leq$ s1.length(), s2.length() ≤ 103
Both the strings are in lowercase.

Code:

```c
#include <stdio.h>
#include <string.h>

int min(int a, int b, int c)
{
    int minimum = a;

    if(b < minimum)
    {
        minimum = b;
    }

    if(c < minimum)
    {
        minimum = c;
    }

    return minimum;
}

int main()
{
    char s1[1000], s2[1000];

    printf("Roll Number : CH.EN.U4CCE24020\n\n");

    printf("Enter first string: ");
    scanf("%s", s1);

    printf("Enter second string: ");
    scanf("%s", s2);

    int len1 = strlen(s1);
    int len2 = strlen(s2);

    int dp[1001][1001];

    // Initialize first row and column
    for(int i = 0; i <= len1; i++)
    {
        dp[i][0] = i;
    }

    for(int j = 0; j <= len2; j++)
    {
        dp[0][j] = j;
    }

    // Fill DP table
    for(int i = 1; i <= len1; i++)
    {
        for(int j = 1; j <= len2; j++)
        {
            if(s1[i - 1] == s2[j - 1])
            {
                dp[i][j] = dp[i - 1][j - 1];
            }
            else
            {
                dp[i][j] = 1 + min(
                    dp[i - 1][j],     // Remove
                    dp[i][j - 1],     // Insert
                    dp[i - 1][j - 1]  // Replace
                );
            }
        }
    }

    printf("\nMinimum operations required: %d\n", dp[len1][len2]);

    return 0;
}
```

Output

```
Roll Number : CH.EN.U4CCE24020

Enter first string: "geek"
Enter second string: "gesek"
```

13. Given a string str of opening and closing brackets '(' and ')' only. The task is to find an equal point. An equal point is an index (0-based) such that the number of closing brackets on the right from that point must be equal to the number of opening brackets before that point.

Examples:

Input: str = "(()))("
Output: 4
Explanation: After index 4, string splits into (()) and))(. Number of opening brackets in the first part is equal to number of closing brackets in the second part.
Input : str = "))"
Output: 2
Explanation: As after 2nd position i.e.)) and "empty" string will be split into these two parts: So, in this number of opening brackets i.e. 0 in the first part is equal to number of closing brackets in the second part i.e. also 0.
Expected Time Complexity: O(n)
Expected Auxiliary Space: O(n)

Constraints:
$1 \leq$ str.size() ≤ 106
str consists of only '(' and ')' brackets.

Code:

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char str[1000000];

    printf("Roll Number : CH.EN.U4CCE24020\n\n");

    printf("Enter the bracket string: ");
    scanf("%s", str);

    int n = strlen(str);

    int openCount = 0;
    int closeCount = 0;

    // Count total closing brackets
    for(int i = 0; i < n; i++)
    {
        if(str[i] == ')')
        {
            closeCount++;
        }
    }

    int equalPoint = n;

    for(int i = 0; i < n; i++)
    {
        if(str[i] == '(')
        {
            openCount++;
        }
        else
        {
            closeCount--;
        }

        if(openCount == closeCount)
        {
            equalPoint = i + 1;
            break;
        }
    }

    printf("\nEqual Point Index: %d\n", equalPoint);

    return 0;
}
```

Output

```
Roll Number : CH.EN.U4CCE24020

Enter the bracket string: "(()))("

Equal Point Index: 4

--------------------------------
Process exited after 9.032 seconds with return value 0
Press any key to continue . . .
```

```
Roll Number : CH.EN.U4CCE24020

Enter the bracket string: "))"

Equal Point Index: 2

--------------------------------
Process exited after 1.901 seconds with return value 0
Press any key to continue . . .
```

14. Given two numbers as strings s1 and s2. Calculate their Product.
Note: The numbers can be negative and You are not allowed to use any built-in function or convert the strings to integers. There can be zeros in the begining of the numbers. You don't need to specify '+' sign in the begining of positive numbers.

Examples:

Input: s1 = "0033", s2 = "2"
Output: "66"
Explanation: 33 * 2 = 66
Input: s1 = "11", s2 = "23"
Output: "253"
Explanation: 11 * 23 = 253
Input: s1 = "123", s2 = "0"
Output: "0"
Explanation: Anything multiplied by 0 is equal to 0.
Constraints:
$1 \leq s1.size() \leq 103$
$1 \leq s2.size() \leq 103$

Code:

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char s1[1005], s2[1005];

    printf("Roll Number : CH.EN.U4CCE24020\n\n");

    printf("Enter first number: ");
    scanf("%s", s1);

    printf("Enter second number: ");
    scanf("%s", s2);

    int sign = 1;

    int i = 0, j = 0;

    // Handle negative sign
    if(s1[0] == '-')
    {
        sign *= -1;
        i = 1;
    }

    if(s2[0] == '-')
    {
        sign *= -1;
        j = 1;
    }

    // Remove leading zeros
    while(s1[i] == '0' && s1[i + 1] != '\0')
    {
        i++;
    }

    while(s2[j] == '0' && s2[j + 1] != '\0')
    {
        j++;
    }

    char *num1 = s1 + i;
    char *num2 = s2 + j;

    int len1 = strlen(num1);
    int len2 = strlen(num2);

    // If any number is 0
    if((len1 == 1 && num1[0] == '0') ||
       (len2 == 1 && num2[0] == '0'))
    {
        printf("\nProduct: 0\n");
        return 0;
    }

    int result[2010] = {0};
```

```c
    printf("\nProduct: ");

    if(sign == -1)
    {
        printf("-");
    }

    int started = 0;

    for(int i = 0; i < len1 + len2; i++)
    {
        if(result[i] != 0)
        {
            started = 1;
        }

        if(started)
        {
            printf("%d", result[i]);
        }
    }

    printf("\n");

    return 0;
}
```

Output

```
Roll Number : CH.EN.U4CCE24020

Enter first number: 033
Enter second number: 2

Product: 66

--------------------------------
Process exited after 9.954 seconds with return value 0
Press any key to continue . . .
```

```
Roll Number : CH.EN.U4CCE24020

Enter first number: 11
Enter second number: 23

Product: 253

--------------------------------
Process exited after 2.091 seconds with return value 0
Press any key to continue . . .
```

```
Roll Number : CH.EN.U4CCE24020

Enter first number: 123
Enter second number: 0

Product: 0

--------------------------------
Process exited after 2.55 seconds with return value 0
Press any key to continue . . .
```

15. A message containing letters A-Z is being encoded to numbers using the following mapping:

'A' -> 1
'B' -> 2
...
'Z' -> 26

You are given a string digits. You have to determine the total number of ways that message can be decoded.

Examples:

Input: digits = "123"
Output: 3
Explanation: "123" can be decoded as "ABC"(1, 2, 3), "LC"(12, 3) and "AW"(1, 23).
Input: digits = "90"
Output: 0
Explanation: "90" cannot be decoded, as it's an invalid string and we cannot decode '0'.
Input: digits = "05"
Output: 0
Explanation: "05" cannot be mapped to "E" because of the leading zero ("5" is different from "05"), the string is not a valid encoding message.
Constraints:
$1 \leq$ digits.size() ≤ 103

Code:

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char digits[1005];

    printf("Roll Number : CH.EN.U4CCE24020\n\n");

    printf("Enter the encoded digit string: ");
    scanf("%s", digits);

    int n = strlen(digits);

    // Invalid if starts with 0
    if(digits[0] == '0')
    {
        printf("\nTotal number of decodings: 0\n");
        return 0;
    }

    int dp[1005];

    dp[0] = 1;
    dp[1] = 1;

    for(int i = 2; i <= n; i++)
    {
        dp[i] = 0;

        // Single digit decoding
        if(digits[i - 1] > '0')
        {
            dp[i] += dp[i - 1];
        }

        // Two digit decoding
        int twoDigit =
            (digits[i - 2] - '0') * 10 +
            (digits[i - 1] - '0');

        if(twoDigit >= 10 && twoDigit <= 26)
        {
            dp[i] += dp[i - 2];
        }
    }

    printf("\nTotal number of decodings: %d\n", dp[n]);

    return 0;
}
```

Output

```
Roll Number : CH.EN.U4CCE24020

Enter the encoded digit string: 12

Total number of decodings: 2

--------------------------------
Process exited after 1.517 seconds with return value 0
Press any key to continue . . .
```

```
Roll Number : CH.EN.U4CCE24020

Enter the encoded digit string: 90

Total number of decodings: 0

--------------------------------
Process exited after 1.266 seconds with return value 0
Press any key to continue . . .
```

```
Roll Number : CH.EN.U4CCE24020

Enter the encoded digit string: 05

Total number of decodings: 0

--------------------------------
Process exited after 1.262 seconds with return value 0
Press any key to continue . . .
```

16. Given a string str of length n, find if the string is K-Palindrome or not. A k-palindrome string transforms into a palindrome on removing at most k characters from it.

Example 1:
Input: str = "abcdecba"
n = 8, k = 1
Output: 1
Explaination: By removing 'd' or 'e'
we can make it a palindrome.

Example 2:
Input: str = "abcdefcba"
n = 9, k = 1
Output: 0
Explaination: By removing a single
character we cannot make it a palindrome.

Your Task:
You do not need to read input or print anything. Your task is to complete the function kPalindrome() which takes string str, n, and k as input parameters and returns 1 if str is a K-palindrome else returns 0.

Expected Time Complexity: O(n*n)
Expected Auxiliary Space: O(n*n)

Constraints:
$1 \leq n, k \leq 103$

Code:

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char str[1005];
    int k;

    printf("Roll Number : CH.EN.U4CCE24020\n\n");

    printf("Enter the string: ");
    scanf("%s", str);

    printf("Enter value of k: ");
    scanf("%d", &k);

    int n = strlen(str);

    int dp[1005][1005];

    // Initialize DP table
    for(int i = 0; i < n; i++)
    {
        dp[i][i] = 0;
    }

    // Fill DP table
    for(int length = 2; length <= n; length++)
    {
        for(int i = 0; i < n - length + 1; i++)
        {
            int j = i + length - 1;

            if(str[i] == str[j])
            {
                dp[i][j] = (i + 1 <= j - 1) ? dp[i + 1][j - 1] : 0;
            }
            else
            {
                int left = dp[i + 1][j];
                int right = dp[i][j - 1];

                if(left < right)
                {
                    dp[i][j] = 1 + left;
                }
                else
                {
                    dp[i][j] = 1 + right;
                }
            }
        }
    }

    if(dp[0][n - 1] <= k)
    {
        printf("\nOutput: 1\n");
    }
    else
    {
        printf("\nOutput: 0\n");
    }

    return 0;
}
```

Output

```
Roll Number : CH.EN.U4CCE24020

Enter the string: "abcdecba"
Enter value of k: 1

Output: 1

--------------------------------
Process exited after 17.93 seconds with return value 0
Press any key to continue . . .
```

```
Roll Number : CH.EN.U4CCE24020

Enter the string: "abcdefcba"
Enter value of k: 1

Output: 0

--------------------------------
Process exited after 12.56 seconds with return value 0
Press any key to continue . . .
```

17. Given two strings txt and pat, return the 0-based index of the first occurrence of the substring pat in txt. If pat is not found, return -1.
Note: You are not allowed to use the inbuilt function.

Examples :

Input: txt = "GeeksForGeeks", pat = "Fr"
Output: -1
Explanation: "Fr" is not present in the string "GeeksForGeeks" as substring.
Input: txt = "GeeksForGeeks", pat = "For"
Output: 5
Explanation: "For" is present as substring in "GeeksForGeeks" from index 5 (0 based indexing).
Input: txt =a "GeeksForGeeks", pat = "gr"
Output: -1
Explanation: "gr" is not present in the string "GeeksForGeeks" as substring.
Constraints:
$1 \leq$ txt.size(),pat.size() ≤ 1000

Code:

```c
#include <string.h>

int main()
{
    char txt[1005], pat[1005];

    printf("Roll Number : CH.EN.U4CCE24020\n\n");

    printf("Enter the main string: ");
    scanf("%1000s", txt);

    printf("Enter the pattern string: ");
    scanf("%1000s", pat);

    int n = strlen(txt);
    int m = strlen(pat);

    int found = -1;

    // Check each possible starting index
    for(int i = 0; i <= n - m; i++)
    {
        int j;

        for(j = 0; j < m; j++)
        {
            if(txt[i + j] != pat[j])
            {
                break;
            }
        }

        // Pattern found
        if(j == m)
        {
            found = i;
            break;
        }
    }

    printf("\nOutput: %d\n", found);

    return 0;
}
```

Output

```
Roll Number : CH.EN.U4CCE24020

Enter the main string:  "GeeksForGeeks"
Enter the pattern string: "Fr"

Output: -1

--------------------------------
Process exited after 23.98 seconds with return value 0
Press any key to continue . . .
```

```
Roll Number : CH.EN.U4CCE24020

Enter the main string: "GeeksForGeeks"
Enter the pattern string: "For"

Output: -1

--------------------------------
Process exited after 17.55 seconds with return value 0
Press any key to continue . . .
```

```
Roll Number : CH.EN.U4CCE24020

Enter the main string: "GeeksForGeeks"
Enter the pattern string: "gr"

Output: -1

--------------------------------
Process exited after 19.25 seconds with return value 0
Press any key to continue . . .
```

18. Given a string s consisting of opening and closing parenthesis '(' and ')'. Find the length of the longest valid parenthesis substring.

A parenthesis string is valid if:

For every opening parenthesis, there is a closing parenthesis.
The closing parenthesis must be after its opening parenthesis.
Examples :

Input: s = ")()())"
Output: 4
Explanation: The longest valid parenthesis substring is "()()".
Input: s = "())()"
Output: 2
Explanation: The longest valid parenthesis substring is "()".
Constraints:
$1 \leq s.size() \leq 106$
s consists of '(' and ')' only

Code:

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char s[1000005];

    printf("Roll Number : CH.EN.U4CCE24020\n\n");

    printf("Enter the parenthesis string: ");
    scanf("%1000000s", s);

    int n = strlen(s);

    int stack[1000005];
    int top = -1;

    // Push initial index
    stack[++top] = -1;

    int maxLength = 0;

    for(int i = 0; i < n; i++)
    {
        if(s[i] == '(')
        {
            stack[++top] = i;
        }
        else
        {
            top--;

            if(top == -1)
            {
                stack[++top] = i;
            }
            else
            {
                int length = i - stack[top];

                if(length > maxLength)
                {
                    maxLength = length;
                }
            }
        }
    }

    printf("\nLength of longest valid parenthesis substring: %d\n", maxLength);

    return 0;
}
```

Output

```
Roll Number : CH.EN.U4CCE24020

Enter the parenthesis string: ")()())"

Length of longest valid parenthesis substring: 4

--------------------------------
Process exited after 1.45 seconds with return value 0
Press any key to continue . . .
```

```
Roll Number : CH.EN.U4CCE24020

Enter the parenthesis string: "())()"

Length of longest valid parenthesis substring: 2

--------------------------------
Process exited after 1.155 seconds with return value 0
Press any key to continue . . .
```

19. Given a string s and a string dictionary d, find the longest string in the dictionary that can be formed by deleting some characters of the given string. If there are more than one possible results, return the longest word with the smallest lexicographical order. If there is no possible result, return the empty string.

Examples :

Input: d = {"ale", "apple", "monkey", "plea"}, s = "abpcplea"
Output: "apple"
Explanation: After deleting "b", "c", "a" S became "apple" which is present in d.
Input: d = {"a", "b", "c"}, s = "abpcplea"
Output: "a"
Explanation: After deleting "b", "p", "c", "p", "l", "e", "a" S became "a" which is present in d.
Expected Time Complexity: O(n*x)
Expected Auxiliary Space: O(x) where x is the length of the string in d.

Constraints:
$1 \leq n,x \leq 103$

Code:

```c
#include <stdio.h>
#include <string.h>

int isSubsequence(char s[], char word[])
{
    int i = 0, j = 0;

    while(s[i] != '\0' && word[j] != '\0')
    {
        if(s[i] == word[j])
        {
            j++;
        }

        i++;
    }

    return word[j] == '\0';
}

int main()
{
    int n;

    static char s[1005];
    static char d[1005][1005];
    static char result[1005] = "";

    printf("Roll Number : CH.EN.U4CCE24020\n\n");

    printf("Enter number of dictionary words: ");
    scanf("%d", &n);

    printf("Enter dictionary words:\n");

    for(int i = 0; i < n; i++)
    {
        scanf("%1000s", d[i]);
    }

    printf("Enter the main string: ");
    scanf("%1000s", s);

    for(int i = 0; i < n; i++)
    {
        if(isSubsequence(s, d[i]))
        {
            if(strlen(d[i]) > strlen(result))
            {
                strcpy(result, d[i]);
            }
            else if(strlen(d[i]) == strlen(result))
            {
                if(strcmp(d[i], result) < 0)
                {
                    strcpy(result, d[i]);
                }
            }
        }
    }

    printf("\nLongest possible word: %s\n", result);

    return 0;
}
```

Output

```
Roll Number : CH.EN.U4CCE24020

Enter number of dictionary words: 4
Enter dictionary words:
ale
apple
monkey
plea
Enter the main string: abpcplea

Longest possible word: apple

--------------------------------
Process exited after 17.79 seconds with return value 0
Press any key to continue . . .
```

```
Roll Number : CH.EN.U4CCE24020

Enter number of dictionary words: 3
Enter dictionary words:
a
b
c
Enter the main string: abpcplea

Longest possible word: a

--------------------------------
Process exited after 8.388 seconds with return value 0
Press any key to continue . . .
```

20. Given strings s1, s2, and s3, find whether s3 is formed by an interleaving of s1 and s2.
Interleaving of two strings s1 and s2 is a way to mix their characters to form a new string s3, while maintaining the relative order of characters from s1 and s2. Conditions for interleaving:

Characters from s1 must appear in the same order in s3 as they are in s1.
Characters from s2 must appear in the same order in s3 as they are in s2.
The length of s3 must be equal to the combined length of s1 and s2.
Examples :

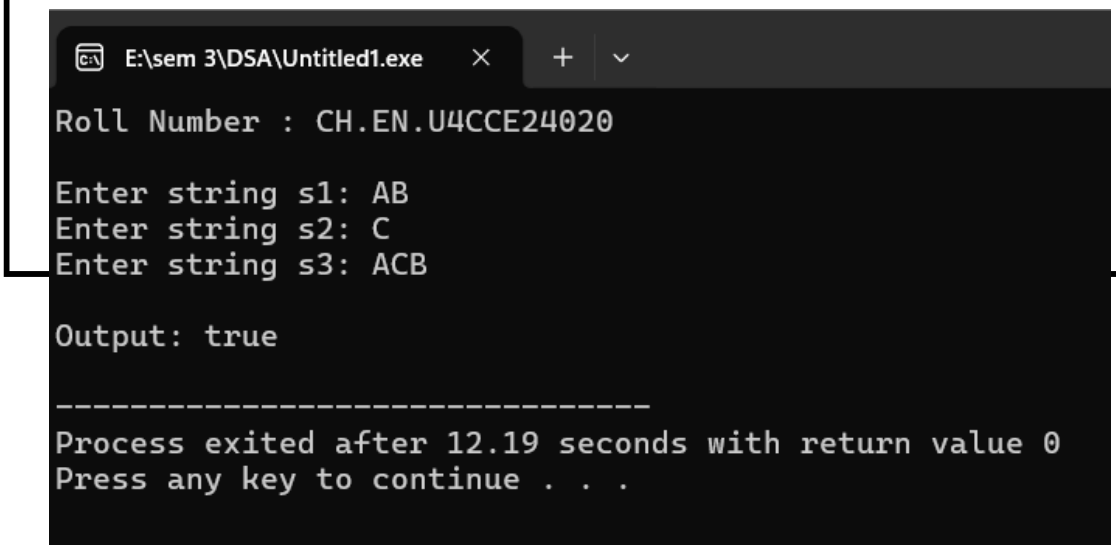
Input: s1 = "AAB", s2 = "AAC", s3 = "AAAABC"
Output: true
Explanation: The string "AAAABC" has all characters of the other two strings and in the same order.
Input: s1 = "AB", s2 = "C", s3 = "ACB"
Output: true
Explanation: s3 has all characters of s1 and s2 and retains order of characters of s1.
Input: s1 = "YX", s2 = "X", s3 = "XXY"
Output: false
Explanation: "XXY " is not interleaved of "YX" and "X". The strings that can be formed are YXX and XYX
Constraints:
$1 \leq$ s1.length, s2.length ≤ 300
$1 \leq$ s3.length ≤ 600

Code:

```c
#include <stdio.h>
#include <string.h>

static int dp[305][305];

int main()
{
    char s1[305], s2[305], s3[605];

    printf("Roll Number : CH.EN.U4CCE24020\n\n");

    printf("Enter string s1: ");
    scanf("%304s", s1);

    printf("Enter string s2: ");
    scanf("%304s", s2);

    printf("Enter string s3: ");
    scanf("%604s", s3);

    int len1 = strlen(s1);
    int len2 = strlen(s2);
    int len3 = strlen(s3);

    // Length check
    if(len1 + len2 != len3)
    {
        printf("\nOutput: false\n");
        return 0;
    }

    dp[0][0] = 1;

    for(int i = 0; i <= len1; i++)
    {
        for(int j = 0; j <= len2; j++)
        {
            if(i > 0 &&
               s1[i - 1] == s3[i + j - 1] &&
               dp[i - 1][j])
            {
                dp[i][j] = 1;
            }

            if(j > 0 &&
               s2[j - 1] == s3[i + j - 1] &&
               dp[i][j - 1])
            {
                dp[i][j] = 1;
            }
        }
    }

    if(dp[len1][len2])
    {
        printf("\nOutput: true\n");
    }
    else
    {
        printf("\nOutput: false\n");
    }

    return 0;
}
```

Output

```
Roll Number : CH.EN.U4CCE24020

Enter string s1: AAB
Enter string s2: AAC
Enter string s3: AAAABC

Output: true

--------------------------------
Process exited after 29.32 seconds with return value 0
Press any key to continue . . .
```




```
Roll Number : CH.EN.U4CCE24020

Enter string s1: YX
Enter string s2: X
Enter string s3: XXY

Output: false

--------------------------------
Process exited after 13.09 seconds with return value 0
Press any key to continue . . .
```